ACADEMIC CENTRAL

PRACTICE EXAMINATION

UNIVERSITY · 3RD YEAR

SOFTWARE ENGINEERING

PAPER 1 · PRACTICE A · MARKING GUIDELINE

MARKS: 100

Marking guideline for the Academic Central University · 3rd year Software Engineering paper 1 · practice a.

Written by Academic Central for practice, following third-year Computer Science syllabuses at South African universities (software engineering). It is not a university memorandum.

How to mark: each ✓ is one mark. Accept other correct answers, examples and wording. Diagrams do not have to look exactly like the model answers: give the marks for the correct elements and the correct UML notation.

QUESTION 1

1.1 A ✓
1.2 B ✓
1.3 B ✓
1.4 C ✓
1.5 B ✓
1.6 C ✓
1.7 A ✓
1.8 B ✓
1.9 C ✓
1.10 B ✓

(10)

[10]

QUESTION 2

2.1 The phases (requirements, design, implementation, testing, deployment and maintenance) follow one after the other, and each phase is finished and signed off before the next one starts ✓
Suits: projects with clear, stable requirements ✓ and safety-critical or contract-based projects that need full documentation (or small, well-understood projects) ✓
Weakness: changes are hard and costly once a phase is closed, and the client sees working software only near the end ✓ (4)

2.2 Individuals and interactions over processes and tools ✓
Working software over comprehensive documentation ✓
Customer collaboration over contract negotiation ✓
Responding to change over following a plan ✓ (4)

2.3 A sprint is a fixed time-box (usually 1 to 4 weeks) in which the team builds a usable, potentially releasable increment ✓
Sprint planning: choose the backlog items and plan how to do them (the sprint goal) ✓
Daily scrum: a 15-minute daily meeting to check progress towards the goal and raise obstacles ✓
Sprint review: show the increment to stakeholders and get feedback; update the product backlog ✓
Sprint retrospective: the team looks at how it worked and agrees on improvements ✓ (5)

2.4 Every loop of the spiral includes an explicit risk-analysis step ✓, and prototypes are built to reduce the biggest risks before the team commits to more work and money ✓ (2)

[15]

QUESTION 3

3.1 Any THREE, each with a sensible interest: (3)

Students: order quickly and skip the queue ✓

Kiosk staff / owners: see clear orders and sell more ✓

University administration or IT: a secure system that links to student cards ✓

(Also: the payment / finance office, app developers.)

3.2 Functional, any FOUR: (6)

The system shall let a student log in with a student number and password ✓

The system shall show the menu and prices of each kiosk ✓

The system shall let a student place an order and pay from the student-card balance ✓

The system shall let kiosk staff mark an order as ready, and then notify the student ✓

(Also: administrators add kiosks and update menus; a student can see order history.)

Non-functional, any TWO with the type:

Performance: the app handles at least 2 000 orders per hour at lunchtime, and each page loads within 2 seconds ✓

Security: payment data is sent over HTTPS and only logged-in users can order ✓

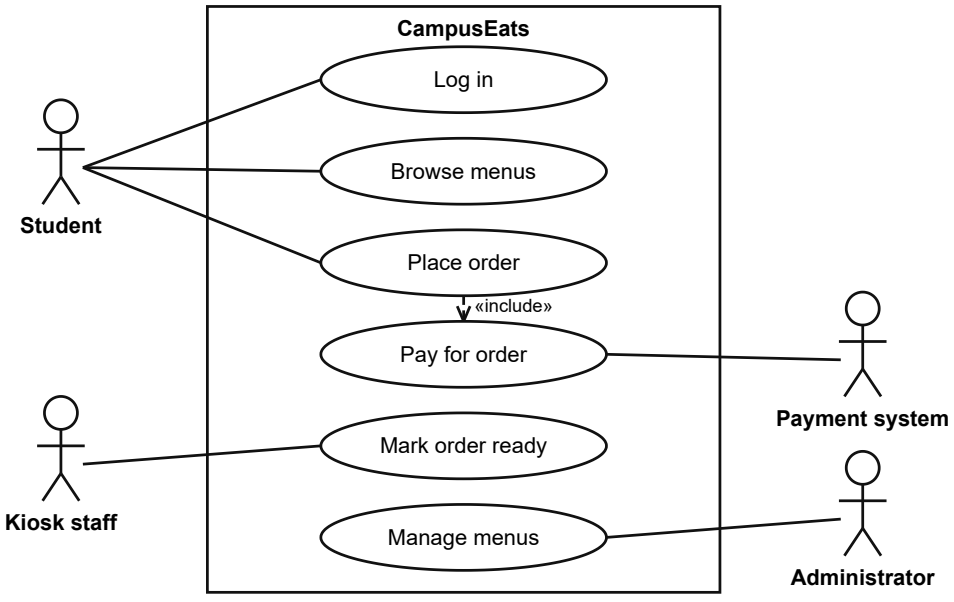
(Also: availability 99% from 07:00 to 18:00; usability; works on Android and iOS.)

3.3 As a student, I want to get a notification when my order is ready, so that I can collect my food without waiting at the kiosk ✓✓ (role, goal and benefit) (4)

Given my order has been paid, when the kiosk staff mark it as ready, then I receive a notification with my order number ✓

Given my order is ready, when I show my order number at the kiosk, then the order is marked as collected ✓

3.4



System boundary with all four actors ✓ at least five suitable use cases ✓✓

correct actor–use case links ✓ a correct «include» ✓

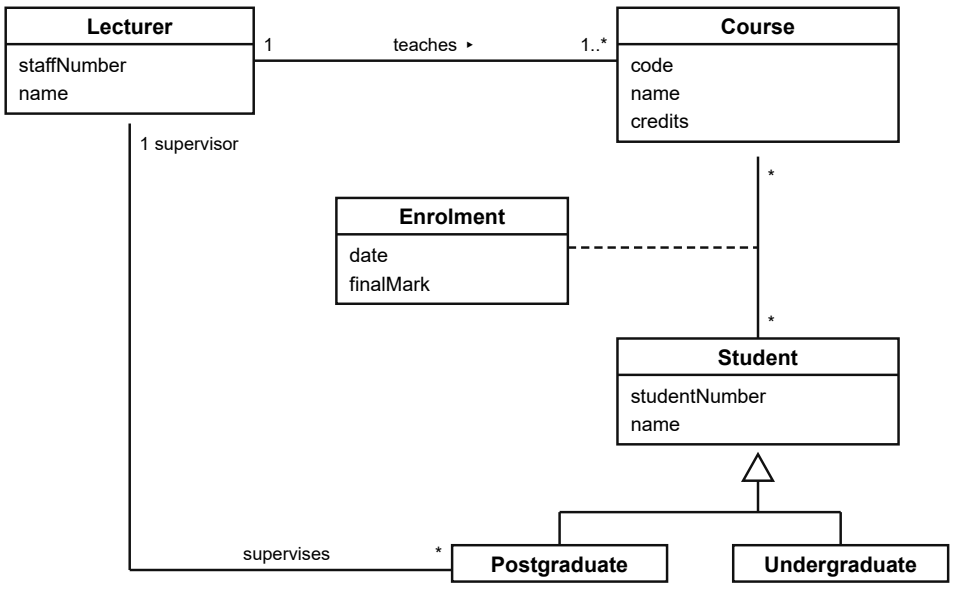
(5)

3.5 Any TWO: interviews ✓, questionnaires or surveys ✓, observation, workshops or focus groups, prototyping, studying documents (2)

[20]

QUESTION 4

4.1



Classes with attributes ✓✓ teaches 1 to 1..* ✓ enrolment many-to-many ✓ with date and mark (association class or Enrolment class) ✓ generalisation with hollow triangle ✓✓ supervises ✓ (8)

4.2 With low coupling, modules depend on each other only through small, clear interfaces ✓, so a change inside one module does not force changes in others. E.g. if the payment module is replaced, the ordering module that only calls pay() does not change ✓ (2)

4.3.1 Observer pattern (publish–subscribe) ✓ (1)

4.3.2 The weather station is the subject: it keeps a list of observers and has attach() and detach() methods ✓ (3)
Each display is an observer that implements an update() method from a common Observer interface ✓
When a new reading arrives, the subject's notify() calls update() on every observer in the list, passing the reading (push) or letting them fetch it (pull) ✓

4.3.3 New displays can be added without changing the weather station class (loose coupling / open–closed) ✓ (1)

4.4 Single responsibility: a class should have only one reason to change ✓ (5)
Open–closed: classes are open for extension but closed for modification ✓
Liskov substitution: a subclass object must be usable wherever its superclass is expected ✓
Interface segregation: many small, specific interfaces are better than one large one ✓
Dependency inversion: depend on abstractions (interfaces), not on concrete classes ✓

[20]

QUESTION 5

5.1.1 Invalid: age < 0, e.g. −5 ✓ (5)
Valid: 0 to 12, e.g. 7 ✓
Valid: 13 to 59, e.g. 30 ✓
Valid: 60 to 120, e.g. 75 ✓
Invalid: age > 120, e.g. 150 ✓
(Also accept a non-integer input such as "abc" or 12.5 as an extra invalid partition.)

5.1.2 −1 and 0; 12 and 13 ✓ 59 and 60 ✓ 120 and 121 ✓ (3)

5.2.1 V(G) = number of decisions + 1 ✓ (3)
Decisions: the first if, the second if and the elif = 3 ✓
V(G) = 3 + 1 = 4 ✓
(Accept 5 if the "or" is counted as a separate condition.)

5.2.2 Four values, e.g. (3)
−5: first if true ("invalid") ✓
80: first if false, second if true ("distinction") ✓
60: elif true ("pass") and 30: elif false ("fail") ✓

5.3 Black-box testing designs tests from the specification only, without looking at the code ✓; white-box testing designs tests from the code's internal structure, e.g. to cover every branch ✓ (2)

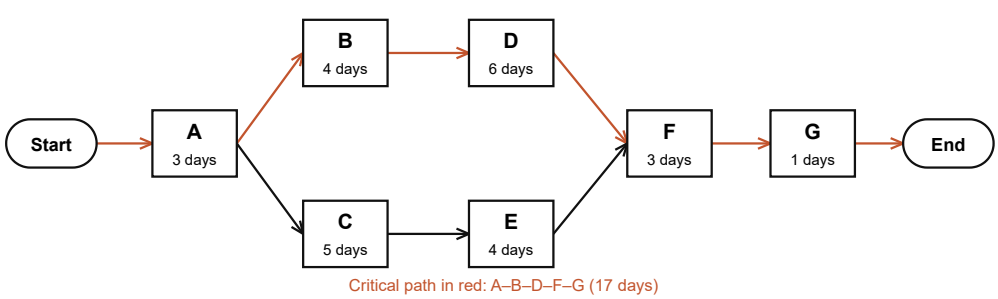
5.4 Unit testing: tests one function or class on its own ✓ (3)
Integration testing: tests that units work correctly together through their interfaces ✓
System testing: tests the whole, complete system against its requirements ✓

5.5 Red (write a failing test), green (write just enough code to pass), refactor (clean up the code) ✓ (1)

[20]

QUESTION 6

6.1.1



Start and end ✓ correct dependencies (B and C after A; F after D and E) ✓
durations shown ✓ (3)

6.1.2 A–B–D–F–G = 3 + 4 + 6 + 3 + 1 = 17 days; A–C–E–F–G = 3 + 5 + 4 + 3 + 1 = 16 days ✓ (3)
Critical path: A–B–D–F–G ✓
Shortest finishing time: 17 days ✓

6.1.3 E can start on day 8 and finish on day 12 at the earliest; F can only start on day 13 (after D), so E may finish as late as day 13 ✓ (2)
Slack of E = 13 − 12 = 1 day ✓

6.2 Corrective maintenance fixes faults that users find, e.g. a wrong total on an invoice ✓; perfective maintenance improves the software or adds features users ask for, e.g. a faster search or a dark mode ✓ (2)

6.3 Benefits, any TWO: the full history of changes is kept and any version can be restored ✓; team members can work on branches at the same time and merge their work ✓; it shows who changed what (3)
A merge conflict happens when two people changed the same lines of a file in different ways, so Git cannot combine them by itself and a person must choose ✓

6.4 Any TWO risks with a way to reduce each, e.g. (2)
Requirements keep changing: use short iterations and agree changes with the product owner ✓
A key developer leaves: share knowledge through pair programming, code reviews and documentation ✓
(Also: the deadline is too short: estimate carefully and plan a buffer; new technology fails: build a prototype first.)

[15]

TOTAL: 100

END OF MARKING GUIDELINE